System Design Foundations and Concepts

🚀 System Design Fundamentals

- What is a System?
  - A collection of components working together to solve use cases towards a common goal.
- Design
  - Choosing and integrating components based on functional and non-functional requirements.
  - Functional: Features and operations of the system.
  - Non-functional: Performance metrics like latency, scalability, availability, security.
- Example: Alien Bank Case Study
  - Single cashier handling slow requests – vertical scaling by improving cashier skills/tools.
  - Adding more cash counters – horizontal scaling with centralized data to avoid inconsistencies.
  - Load balancer (middleman) to route customers to underutilized counters.

🌐 Communication Protocols & APIs

- DNS
  - Translates domain names to IP addresses via a hierarchical query process: root → TLD → authoritative nameserver.
  - Caching at multiple levels improves resolution time.
- APIs
  - Interface between client and server, or between different applications.
  - Common types: REST (JSON), SOAP (XML), GraphQL, gRPC, WebSockets.
  - RESTful API methods: GET, POST, PUT, PATCH, DELETE.
  - Endpoints structured as resources and nested paths for relationships.

⚙️ Caching Strategies

- Purpose of Cache
  - Stores frequently accessed data to reduce database load and lower latency.
  - Cache hit (data found) vs cache miss (fall back to DB).
- Cache Eviction Policies
  - LRU (Least Recently Used)
  - MRU (Most Recently Used)
  - LFU (Least Frequently Used)
  - FIFO (First In First Out)
  - LIFO (Last In First Out)
- Cache Write Strategies
  - Read-Through Cache (read via cache, write to DB)
  - Write-Through Cache (write to cache and DB synchronously)
  - Write-Around Cache (write directly to DB, read from cache)
  - Write-Back Cache (async DB writes, write to cache first)

🔔 Message Queues & Async Processing

- Sync vs Async Requests
  - Sync: immediate response needed (e.g., payment processing).
  - Async: can process later, decouples producer and consumer (e.g., email notifications).
- Message Queue Roles
  - Producers publish messages to the queue.
  - Consumers subscribe/pull messages for processing.
  - Ensures durability, ordering (FIFO or priority), fault handling (dead letter queues).
- Use Cases & Limitations
  - Suitable for workload buffering, decoupling services, deferred processing.
  - Not ideal for real-time or where immediate acknowledgements required.

📹 Case Study: Video Streaming System Design

- Challenges in Video Streaming
  - Large file sizes, variable network bandwidth, device resolutions.
  - Need for segmenting video into chunks and adaptive bitrate streaming.
- Core Architecture
  - Source video encoding and segmenting service.
  - Priority message queues for encoding/transcoding quality versions.
  - Distributed CDN for content delivery optimized by geography.
  - Client-side adaptive streaming based on real-time bandwidth.
- Performance Considerations
  - Data chunk sizes vs network speed.
  - Resolution switching based on device and connection.
  - Segment caching to reduce latency and buffering.

🧩 Introduction to System Design

- System design solves how software handles millions of simultaneous users
- It goes beyond code correctness to scalability and reliability under heavy load.
- Foundational concepts include system components, communication protocols, database types, scaling strategies, and fault tolerance.
- Importance in Interviews
  - Most tech interviews include system design questions, requiring understanding trade-offs and architecture decisions.
- Components Overview
  - Client applications (web, mobile) interact via APIs.
  - Application code manages logic and talks to databases.
  - Databases store persistent data either in SQL or NoSQL types.
  - Caches improve frequent data access speed.
  - Load balancers distribute requests across servers.
  - Message queues handle asynchronous, decoupled communication.
  - Monitoring ensures system health and performance.

🗄️ Databases and Data Intensity

- SQL vs NoSQL
  - SQL: Structured schema, relational tables with joins, strong consistency.
  - NoSQL: Schema-less, flexible data models (key-value, document, graph), easier horizontal scaling.
- Data-Intensive vs Compute-Intensive Systems
  - Data-intensive focus on storing/retrieving large data volumes efficiently (e.g., Instagram, WhatsApp).
  - Compute-intensive focus on heavy calculations on smaller data sets (e.g., simulations, ML model training).

⚖️ Scaling and Load Balancing

- Vertical vs Horizontal Scaling
  - Vertical: Upgrade existing machine's capacity.
  - Horizontal: Add more machines/servers.
- Load Balancer Functions
  - Distributes incoming requests among servers using algorithms like round robin, least connections, weighted round robin, geo-based, IP hash.
  - Performs health checks actively or passively to route traffic only to healthy servers.
  - Hybrid algorithms and dynamic routing common in real-world systems.

🔁 Replication, Partitioning & CAP Theorem

- Replication
  - Copies data across multiple servers for availability and fault tolerance.
  - Single leader: one primary server replicates to followers (sync or async).
  - Multi-leader: multiple writable leaders, potential conflict resolution needed.
  - Leaderless: no primary, quorum helps decide consistency.
- Partitioning (Sharding)
  - Data split across nodes to handle large volumes beyond single server capacity.
  - Key-based, hash-based, and index-based partitioning methods.
  - Challenge: hotspot nodes with uneven load distribution mitigated by dynamic repartitioning.
- CAP Theorem
  - Systems can guarantee at most two of: Consistency, Availability, Partition tolerance.
  - CP systems prefer data accuracy over availability.
  - AP systems prefer availability over perfect consistency.
  - CA feasible only without partition (single node).

⚠️ Fault Tolerance and Monitoring

- Fault Types
  - Hardware faults: random, physical failures, mitigated by redundancy.
  - Software faults: bugs, misconfigurations, deterministic and fixable by testing.
  - Human faults: unpredictable, require discipline and processes.
- Monitoring & Observability
  - Track API throughput, error rates, latency percentiles (P50, P90, P99).
  - Machine stats: CPU, memory, I/O usage.
  - Active and passive health checks of services.
  - Log aggregation and alerting for rapid issue detection and resolution.

[NoteGPT] AI Mind Map Generator